注目という発想

Attention の直感 — どこを見るかを、データに決めさせる

rkskmt

まずは自分で読んでみる

クイズ

次の（　）に入る言葉は？

> 吾輩は猫である。名前はまだ（　）。

- ほぼ全員が「無い」と答える。では、答えるとき**どこを見た**？
- 直前の「まだ」だけではない。少し離れた「名前は」に目が飛んだはず
- **読むとは、関係する場所に視線を飛ばすこと** — 今日はこれを数式にする

窓のやり方では届かない

- 前回の窓方式：直前 n 文字を**位置で**固定して見る
- 「名前はまだ」の「名前」は9文字前 — 窓5では存在すらしない
- 窓を広げれば届くが、過学習と暗記が待っていた

	窓（前回）	視線（今日）
見る範囲	直前 n 文字で固定	**全部**
見る強さ	全部同じ	**関係の深さで変える**
決め方	人間が n を決める	**データから学習**

道具はもう持っている：重み付き平均

- 文字は埋め込みで**ベクトル**になっている（ことばの地図の回）
- 複数のベクトルを「混ぜる」自然な方法＝**重み付き平均**（weighted average）

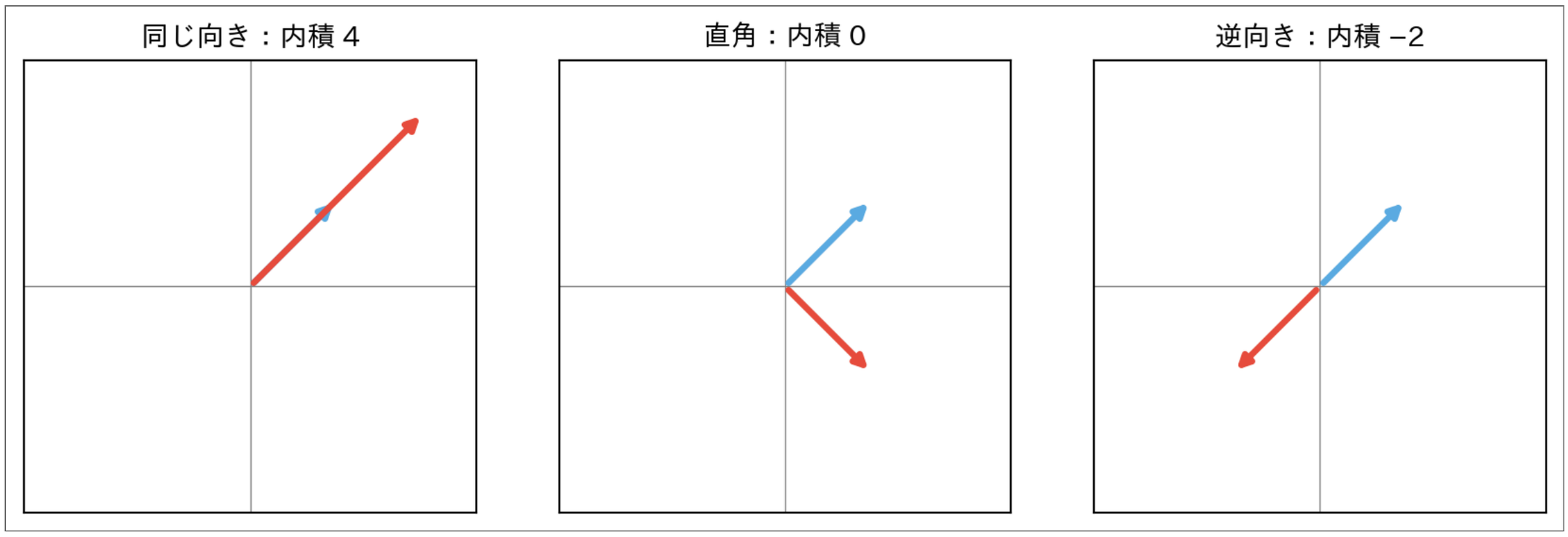
$$\text{混ぜた結果} = 0.5 \cdot \vec{v}_1 + 0.25 \cdot \vec{v}_2 + 0.25 \cdot \vec{v}_3$$

- 重みは**合計1**の割合。大事な文字に大きな重みを置けば「そこを強く見た」ことになる
- 問題はただ1つ：**その重みを誰がどう決めるか**

内積 — 「似ている度」を1つの数に

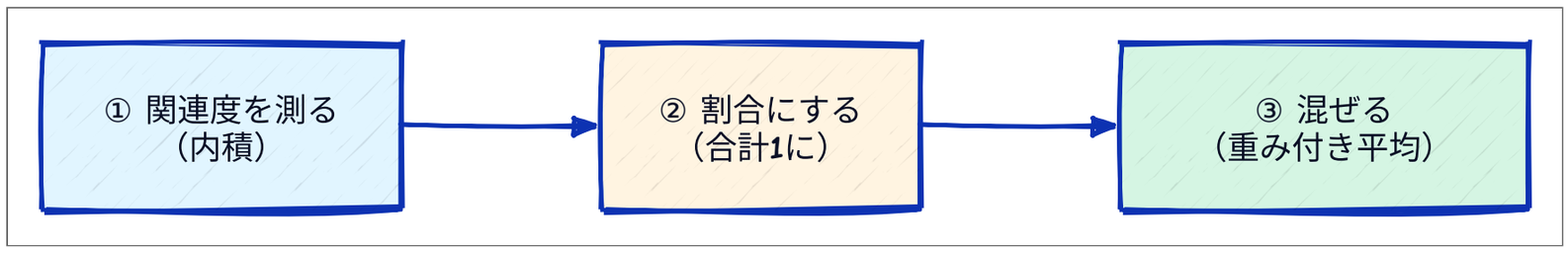
- 2本のベクトルの**内積**（dot product）＝ **成分どうしを掛けて、足すだけ**

$$(1,1)\cdot(2,2) = 1\times 2 + 1\times 2 = 4$$



- **向きが揃うほど大きくなる** — 「似ている度」を1つの数で測れる物差し
- ことばの地図の回の「近い文字＝似た使われ方」と同じ発想の親戚。今日はこの物差しで「関連の深さ」を測る

3ステップの表計算



- 各文字に2種類のベクトルを持たせる：
  - → **持ってる情報** $\vec{v}$：混ぜられる中身
  - → **名札** $\vec{k}$：「私はこういう情報を持っています」の看板
- いま予測しようとしている場所は「**探しもの**」$\vec{q}$ を1本持つ
- 関連度 ＝ **名札と探しものの内積** — さっきの「似ている度」の物差しをそのまま使う

手計算1往復 — 「名前はまだ」で

ミニ文「名前／は／まだ」の3トークン。**「まだ」の場所**から過去を見る。数値は**電卓なしで**埋められるおもちゃサイズにしてある。

トークン	名札 $\vec{k}$	持ってる情報 $\vec{v}$
「名前」	$(1, 1)$	$(1, 0)$
「は」	$(1, 0)$	$(0, 1)$
「まだ」	$(0, 1)$	$(1, 1)$

「まだ」の探しもの：$\vec{q} = (1, 1)$

1. **関連度**（内積）：$\vec{q} \cdot \vec{k}_{\text{名前}} = 1{+}1 = 2$、同様に $\vec{q} \cdot \vec{k}_{\text{は}} = 1$、$\vec{q} \cdot \vec{k}_{\text{まだ}} = 1$
2. **割合**：合計4で割って $(0.5,\ 0.25,\ 0.25)$
3. **混ぜる**：$0.5(1, 0) + 0.25(0, 1) + 0.25(1, 1) = (0.75,\ 0.5)$
   - 「まだ」の探しものと一番似た名札は「名前」→ **「名前」の情報が半分を占めた**
   - 冒頭のクイズで目が「名前」に飛んだ、あの動きを数で再現した1往復

検算 — コードだと3行

Code

```python
import numpy as np

K = np.array([[1.0, 1.0], [1.0, 0.0], [0.0, 1.0]])   # 名札（「名前」「は」「まだ」）
V = np.array([[1.0, 0.0], [0.0, 1.0], [1.0, 1.0]])   # 持ってる情報
q = np.array([1.0, 1.0])                               # 「まだ」の探しもの

scores = K @ q                       # ① 関連度
weights = scores / scores.sum()      # ② 割合にする
print("重み:", weights)
print("混ぜた結果:", weights @ V)  # ③ 重み付き平均
```

Result

```
重み: [0.5  0.25 0.25]
混ぜた結果: [0.75 0.5 ]
```

- 手計算と一致。**注意の中身は、内積と足し算だけ**

割合化には落とし穴がある

> **クイズ**
>
> 関連度が $(2, -1, -1)$ になったら？「合計で割る」方式で割合を作ってみると……

- 合計は 0 — **ゼロ割り崩壊**。負の重み「−∞%見る」も意味不明
- 欲しい性質：常に正・合計1・大きいスコアほど強調
- その専用部品はすでに使っている — **「学習で当てる」の softmax**（e^s を取って合計で割る）

Code

```python
import numpy as np

def softmax(s):
    e = np.exp(s)
    return e / e.sum()

print(softmax(np.array([2.0, 1.0, 1.0])))     # さっきの例
print(softmax(np.array([2.0, -1.0, -1.0])))   # 負が混ざっても安全
```

Result

```
[0.57611688 0.21194156 0.21194156]
[0.909443  0.0452785 0.0452785]
```

- 以後、②の割合化は softmax で行う。さっきの例なら重み $(0.58, 0.21, 0.21)$ — 1位がより強調される

これが Attention

- いまの3ステップ＝**注意**（**attention**）機構。直訳どおり「どこに注意を向けるか」

$\mathrm{Attention} = \mathrm{softmax}(\text{関連度})$ で混ぜた重み付き平均

- 窓と違い、**遠くても関係が深ければ強く見る**。関係が薄ければ隣でもほぼ無視
- 「名前はまだ（　）」問題：「まだ」の場所の探しもの $\overrightarrow{q}$ が「名前」の名札 $\overrightarrow{k}$ と強く共鳴すれば、**9文字の距離は関係ない**（さっきの手計算はそのミニチュア）

> **ノート**
>
> 残る疑問はただ1つ：**名札・持ってる情報・探しものの中身は誰が書くのか。** 答えは次回 — もちろん、学習で作らせる。

Tweak 探しものを変えると視線が動く



- **検算のコード** の `q` を変えて、softmax 版で実行

1	`q = np.array([1.0, 0.0])`	「名前」と「は」の2人が同点1位になる
2	`q = np.array([0.0, 1.0])`	視線は誰に移る？（内積を暗算してから実行）
3	`q = np.array([10.0, 10.0])`	スコアが大きくなると softmax の重みはほぼ一極集中する

ここで観察

- $q = (1, 0)$：スコア $(1, 1, 0)$ → 重み $(0.42, 0.42, 0.16)$ — **同点なら山分け**
- ==視線（重み）は q と k の組み合わせだけで決まる==。ベクトルが変われば、同じ仕組みのまま視線が変わる — 「見る場所の学習」への伏線

まとめ

- 注意＝**3ステップの表計算**：関連度（内積）→ 割合（softmax）→ 重み付き平均
- 窓は「位置」で見る、注意は「内容」で見る。遠さは関係ない
- 部品は全部知り合い：内積・softmax・平均。新しい発想は「重みをデータに決めさせる」ことだけ
- 手計算1往復ができた — 中身はもうブラックボックスではない

現在の設計図　各文字 → **関連度 ⇒ 割合 ⇒ 重み付き平均** → 文脈つきベクトル

次回予告

名札 k・情報 v・探しもの q を、埋め込みから**行列1枚ずつ**で作る。 名前だけ有名な **Q・K・V** の正体は、アフィン変換3つ。

練習問題

手計算で解いてから、コードで検算せよ。

問題1：視線を計算する

本編と同じ「名前」「は」「まだ」の名札・情報（$\overrightarrow{k}_1 = (1,1),\ \overrightarrow{k}_2 = (1,0),\ \overrightarrow{k}_3 = (0,1)$、$\overrightarrow{v}_1 = (1,0),\ \overrightarrow{v}_2 = (0,1),\ \overrightarrow{v}_3 = (1,1)$）で、探しものが $\overrightarrow{q} = (0,2)$ のとき：

1. 関連度3つを内積で求める
2. 「合計で割る」方式で割合にする
3. 混ぜた結果のベクトルを求める

問題2：平均は注意の特別な場合

「全部のベクトルの単純平均」も注意の一種であることを示せ：関連度3つが**すべて同じ値**のとき、割合と混ぜた結果がどうなるか手計算で確かめよ（softmax 方式でも同じことを確認）。

解答例

▸ 解答例を見る

Result

```
スコア: [2. 0. 2.] | 割合: [0.5 0.  0.5] | 結果: [1.  0.5]
合計割り: [0.33333333 0.33333333 0.33333333] | softmax: [0.33333333 0.33333333
0.33333333]
単純平均: [0.66666667 0.66666667] = [0.66666667 0.66666667]
```

- 問題2：softmax でもスコアが同じなら均等割り。**「特に見るところがない」とき、注意は自然に平均へ退化する**